

Aula 10

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) Métricas de Software - Teoria	3
2) Métricas de Software - Questões Comentadas - VUNESP	22
3) Métricas de Software - Lista de Questões - VUNESP	25

MÉTRICAS DE SOFTWARE

Conceitos Básicos

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

MÉTRICAS DE SOFTWARE

Medidas quantitativas utilizadas para avaliar características de produtos, processos e projetos de software, fornecendo informações objetivas que auxiliam no planejamento, controle, monitoramento e melhoria das atividades de desenvolvimento e manutenção. Por meio dessas métricas, é possível mensurar atributos como tamanho, complexidade, qualidade, produtividade, confiabilidade, desempenho, custo e prazo, transformando dados coletados em informações úteis para a tomada de decisões. Dessa forma, as métricas contribuem para reduzir a subjetividade das avaliações, permitindo acompanhar a evolução dos sistemas e verificar se os objetivos de qualidade e desempenho estão sendo alcançados.

As métricas de software são medidas quantitativas utilizadas para avaliar produtos, processos e projetos. Elas fornecem informações objetivas que auxiliam na tomada de decisões, reduzindo a dependência de percepções subjetivas. Por meio dessas medições, é possível acompanhar a evolução dos sistemas, identificar problemas, estimar recursos e verificar se os objetivos de qualidade, prazo e custo estão sendo alcançados ao longo do desenvolvimento.

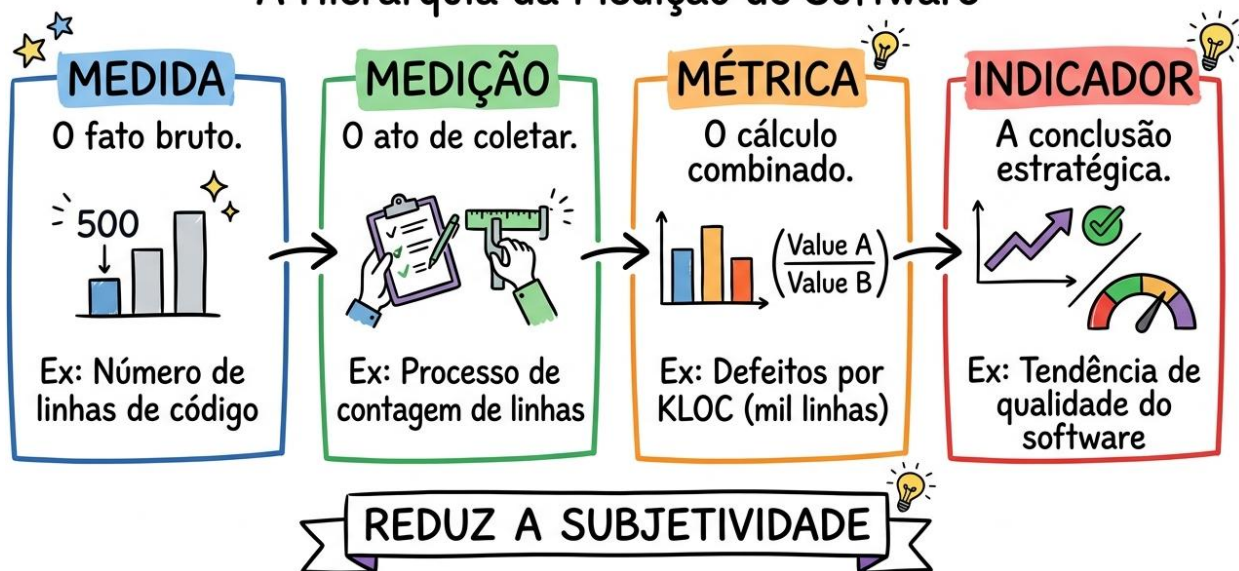
As métricas podem ser classificadas em três grandes categorias. As métricas de produto avaliam características do software, como tamanho, complexidade, qualidade e desempenho. As métricas de processo analisam a eficiência das atividades de desenvolvimento e manutenção. Já as métricas de projeto apoiam o gerenciamento de iniciativas específicas, permitindo monitorar esforço, cronograma, custos e outros indicadores relevantes para o controle do trabalho.

A utilização sistemática dessas métricas favorece o planejamento, a melhoria contínua e a avaliação dos resultados obtidos. Entretanto, nenhuma métrica isolada é capaz de representar toda a complexidade de um software. Por isso, a Engenharia de Software emprega conjuntos de métricas complementares para analisar diferentes atributos, como modularidade, coesão, acoplamento e confiabilidade, obtendo uma visão mais ampla e consistente da qualidade do sistema.

CONCEITOS	DEFINIÇÃO	EXEMPLO
MEDIDA	Valor quantitativo obtido diretamente da observação de uma característica de um produto, processo ou projeto de software.	Número de linhas de código, quantidade de defeitos, número de módulos ou tempo gasto em uma atividade.
MEDIÇÃO	Processo utilizado para coletar e obter medidas de forma sistemática, seguindo critérios e procedimentos definidos.	Utilizar uma ferramenta para contar linhas de código ou registrar o tempo necessário para corrigir um defeito.
MÉTRICA	Medida ou conjunto de medidas combinado para fornecer informações mais significativas sobre um atributo do software.	Densidade de Defeitos, Complexidade Ciclométrica, MTBF e Índice de Manutenibilidade.
INDICADOR	Informação resultante da interpretação de uma ou mais métricas, utilizada para apoiar decisões e avaliações.	Concluir que um projeto está atrasado com base em um SPI menor que 1 ou que a qualidade está adequada porque a densidade de defeitos está abaixo do limite estabelecido.

≡ DOS DADOS À DECISÃO ≡

A Hierarquia da Medição de Software



Classificações das Métricas

Quanto à Obtenção

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Após compreender os conceitos de medida, medição, métrica e indicador, surge uma nova etapa no estudo da medição de software: a organização das diversas métricas existentes. Por que essa organização é necessária? Porque a Engenharia de Software utiliza métricas para avaliar aspectos muito diferentes de um projeto. Algumas estão voltadas para características do produto desenvolvido, outras analisam processos, produtividade, qualidade do código ou comportamento do sistema durante sua execução. Sem uma estrutura de classificação, a seleção e a interpretação dessas métricas tornam-se mais difíceis.

A diversidade de métricas reflete a própria complexidade dos projetos de software. O que exatamente pode ser medido? Dependendo do objetivo da análise, pode-se avaliar a qualidade dos requisitos, a estrutura da arquitetura, a produtividade, a ocorrência de defeitos, o desempenho do sistema ou a confiabilidade operacional. Cada uma dessas perspectivas exige métricas apropriadas, construídas para observar características específicas do ambiente de desenvolvimento ou do produto gerado.

Métricas Diretas

MÉTRICAS DIRETAS

As métricas diretas são aquelas obtidas por meio da observação, contagem ou medição direta de atributos do produto ou do processo de software, sem a necessidade de cálculos derivados. Exemplos incluem linhas de código, número de defeitos encontrados, esforço em horas de trabalho e quantidade de módulos. Essas métricas servem como base para o cálculo de métricas mais elaboradas e fornecem informações objetivas sobre características do projeto ou do software.

As métricas utilizadas na Engenharia de Software podem ser classificadas de diferentes maneiras, e uma das distinções mais importantes envolve as métricas diretas e indiretas. O que caracteriza uma métrica direta? Trata-se de uma medida obtida por observação, contagem ou medição imediata de algum

atributo do software, do processo ou do projeto, sem a necessidade de realizar cálculos adicionais ou combinar diferentes informações. Essas métricas representam os dados brutos coletados diretamente da realidade observada e servem como ponto de partida para análises mais elaboradas.

Por que as métricas diretas são consideradas tão importantes? Porque elas fornecem informações objetivas e de fácil obtenção sobre os elementos avaliados. Em vez de depender de interpretações ou relacionamentos entre múltiplas variáveis, essas métricas registram características observáveis de maneira imediata. Dessa forma, constituem a matéria-prima utilizada na construção de métricas indiretas e indicadores gerenciais voltados ao acompanhamento da qualidade, da produtividade e do desempenho dos projetos.

Dentre as principais métricas diretas, temos: Linhas de Código (LOC), Número de Módulos, Número de Classes, Número de Atributos, Número de Operações, Comprimento de Identificadores, MTTF, MTTR, Tempo de Resposta, Throughput, Uso de Recursos, Número de Scripts de Cenário, Número de Classes-Chave, Número de Classes de Apoio e Número de Subsistemas.

Métricas Indiretas

MÉTRICAS INDIRETAS

As métricas indiretas são obtidas a partir da combinação ou do processamento de uma ou mais medidas diretas. Elas procuram representar atributos mais abstratos da qualidade do software, como confiabilidade, produtividade, qualidade ou manutenibilidade. Exemplos incluem densidade de defeitos, Defect Removal Efficiency (DRE), índice de manutenibilidade e produtividade. Em geral, essas métricas fornecem informações mais relevantes para a tomada de decisões gerenciais.

Nem todas as características relevantes de um software podem ser obtidas por meio de uma simples contagem ou observação direta. Como medir atributos como qualidade, confiabilidade ou manutenibilidade? Para responder a essa necessidade, a Engenharia de Software utiliza métricas indiretas. Essas métricas são construídas a partir da combinação ou interpretação de diferentes medidas básicas e procuram representar características mais complexas do produto, do processo ou do projeto.

O que diferencia uma métrica indireta de uma métrica direta? Enquanto métricas diretas fornecem dados brutos, como linhas de código, quantidade de defeitos ou tempo de resposta, as métricas indiretas transformam essas informações em indicadores mais significativos. Dessa forma, tornam-se capazes de representar atributos que não podem ser observados imediatamente, oferecendo uma visão mais próxima das necessidades de gestão e avaliação da qualidade.

Entre os exemplos mais conhecidos encontram-se: Pontos de Função, Pontos de Casos de Uso, Complexidade Ciclomática, Fan-In/Fan-Out, Coesão, Acoplamento, Densidade de Defeitos, Coberturas de Testes, MTBF, Média de Classes de Apoio, Dívida Técnica, Métricas de Halstead, Razão Comentários/Código, DRE, Taxa de Retrabalho e Produtividade.



Quanto à Coleta

INCIDÊNCIA EM PROVA: BAIXA

Outra forma bastante comum de classificar as métricas de software consiste em analisar o momento em que os dados são coletados, dando origem às chamadas métricas estáticas e métricas dinâmicas. Essa classificação leva em consideração se a avaliação é realizada sem a execução do software ou durante sua execução. Ambas as abordagens são importantes e complementares, pois permitem observar diferentes aspectos da qualidade e do comportamento de um sistema.

Métricas Estáticas

MÉTRICAS ESTÁTICAS

As métricas estáticas são calculadas sem a execução do software, utilizando informações provenientes do código-fonte, da documentação ou do projeto. Elas são empregadas para avaliar características estruturais do sistema, como tamanho, complexidade e organização do código. Exemplos incluem linhas de código, complexidade ciclomática, Fan-In/Fan-Out, acoplamento e coesão.

A avaliação da qualidade de software não depende exclusivamente da execução do sistema. Como identificar problemas antes mesmo que o programa seja colocado em funcionamento? Para essa finalidade, a Engenharia de Software utiliza as métricas estáticas. Essas métricas são obtidas por meio da análise de artefatos produzidos durante o desenvolvimento, como código-fonte, diagramas, modelos e documentação técnica. Como não exigem a execução do software, elas podem ser aplicadas desde as fases iniciais do projeto.

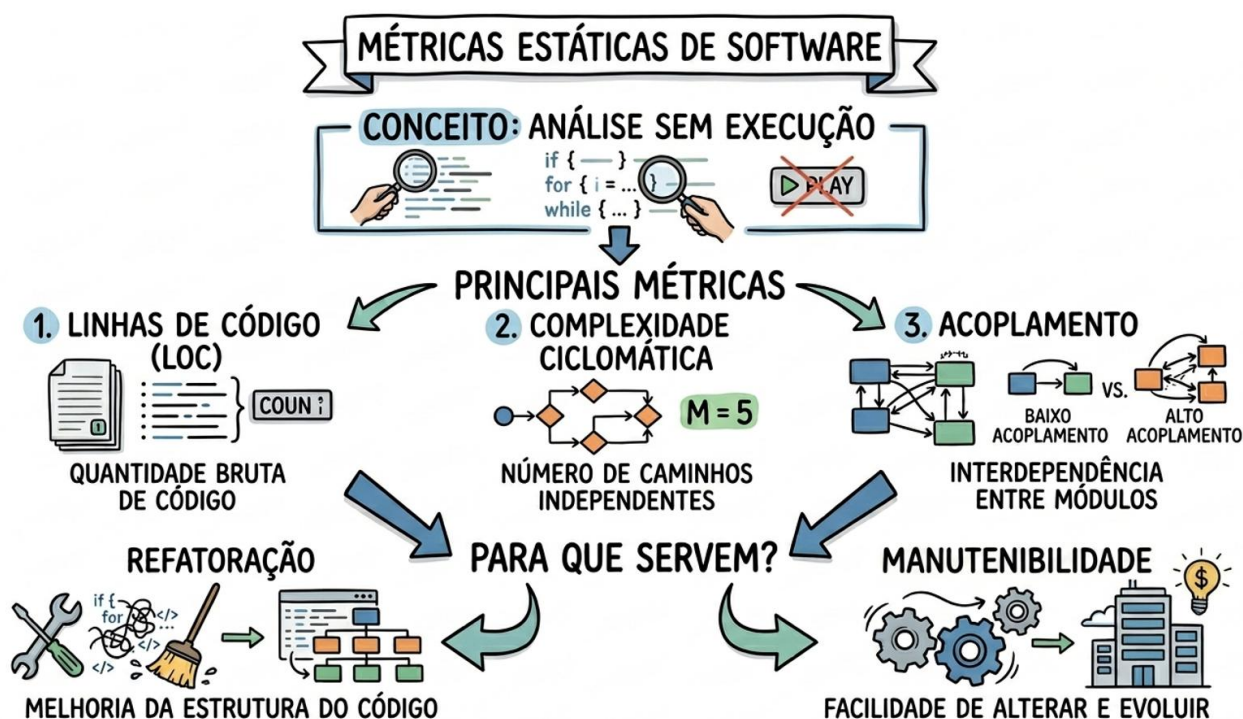
O que exatamente essas métricas procuram avaliar? Seu objetivo é analisar características internas do sistema, fornecendo informações sobre aspectos estruturais que podem influenciar a qualidade do produto final. Ao examinar elementos como organização do código, distribuição de responsabilidades e dependências entre componentes, torna-se possível identificar potenciais dificuldades de manutenção, compreensão e evolução do software antes mesmo da realização dos testes.

Uma das principais vantagens das métricas estáticas está relacionada ao momento em que podem ser utilizadas. Seria necessário aguardar a conclusão do software para avaliar sua qualidade? Não. Como

essas métricas dependem apenas dos artefatos produzidos durante o desenvolvimento, elas permitem identificar problemas potenciais de forma antecipada. Isso possibilita a realização de correções e melhorias quando os custos de modificação ainda são relativamente baixos.

As métricas estáticas demonstram que a qualidade pode ser analisada antes mesmo da execução do sistema. Ao fornecer informações sobre tamanho, complexidade, modularidade e dependências estruturais, essas métricas auxiliam equipes técnicas na identificação de riscos e na melhoria contínua da qualidade interna do software, contribuindo para a construção de sistemas mais compreensíveis, manuteníveis e confiáveis.

Dentre os exemplos mais conhecidos encontram-se: Pontos de Função, Pontos de Casos de Uso, Linhas de Código (LOC), Número de Módulos, Número de Classes, Número de Atributos, Número de Operações, Complexidade Ciclomática, Fan-In/Fan-Out, Coesão, Acoplamento, Índice de Fog, Comprimento de Identificadores, Número de Scripts de Cenário, Número de Classes-Chave, Número de Classes de Apoio, Número Médio de Classes de Apoio por Classe-Chave, Número de Subsistemas, Duplicação de Código, Dívida Técnica, Violações de Estilo e Padrões, Métricas de Halstead e Razão Comentários/Código.



Métricas Dinâmicas

MÉTRICAS DINÂMICAS

As métricas dinâmicas são obtidas basicamente durante a execução do software e refletem seu comportamento em operação. Elas permitem avaliar aspectos como desempenho, confiabilidade e utilização de recursos computacionais. Exemplos incluem tempo de resposta, disponibilidade, consumo de memória, MTTF, MTBF e MTTR.

As métricas dinâmicas permitem avaliar características que somente podem ser observadas quando o software está em funcionamento. Como compreender o comportamento real de um sistema durante sua utilização? Para responder a essa questão, a Engenharia de Software utiliza métricas obtidas durante a execução da aplicação, seja em ambientes de teste, homologação ou produção. Essas medidas fornecem

informações sobre desempenho, confiabilidade, disponibilidade e utilização dos recursos computacionais.

O que diferencia uma métrica dinâmica de uma métrica estática? Enquanto as métricas estáticas analisam artefatos como código-fonte, diagramas e documentação sem executar o sistema, as métricas dinâmicas dependem da observação do software em operação. Dessa forma, elas permitem avaliar como a aplicação realmente se comporta diante das cargas de trabalho, das interações dos usuários e das condições encontradas em ambiente de execução.

As métricas dinâmicas desempenham papel fundamental na avaliação da qualidade externa do software. Por que essa perspectiva é tão importante? Porque um sistema pode apresentar excelente estrutura interna e, ainda assim, revelar problemas apenas quando entra em operação. Ao analisar o comportamento observado durante a execução, essas métricas fornecem informações valiosas sobre desempenho, confiabilidade e experiência dos usuários, permitindo identificar oportunidades de melhoria que dificilmente seriam percebidas apenas por meio da análise estática dos artefatos do sistema.

Dentre os exemplos mais conhecidos, temos: Densidade de Defeitos, Cobertura de Instruções, Cobertura de Decisões, Cobertura Funcional, Cobertura Modular, Cobertura de Código, MTTF (Mean Time To Failure), MTBF (Mean Time Between Failures), MTTR (Mean Time To Repair), Tempo de Resposta, Throughput, Uso de Recursos, DRE (Defect Removal Efficiency), Taxa de Retrabalho e Produtividade.



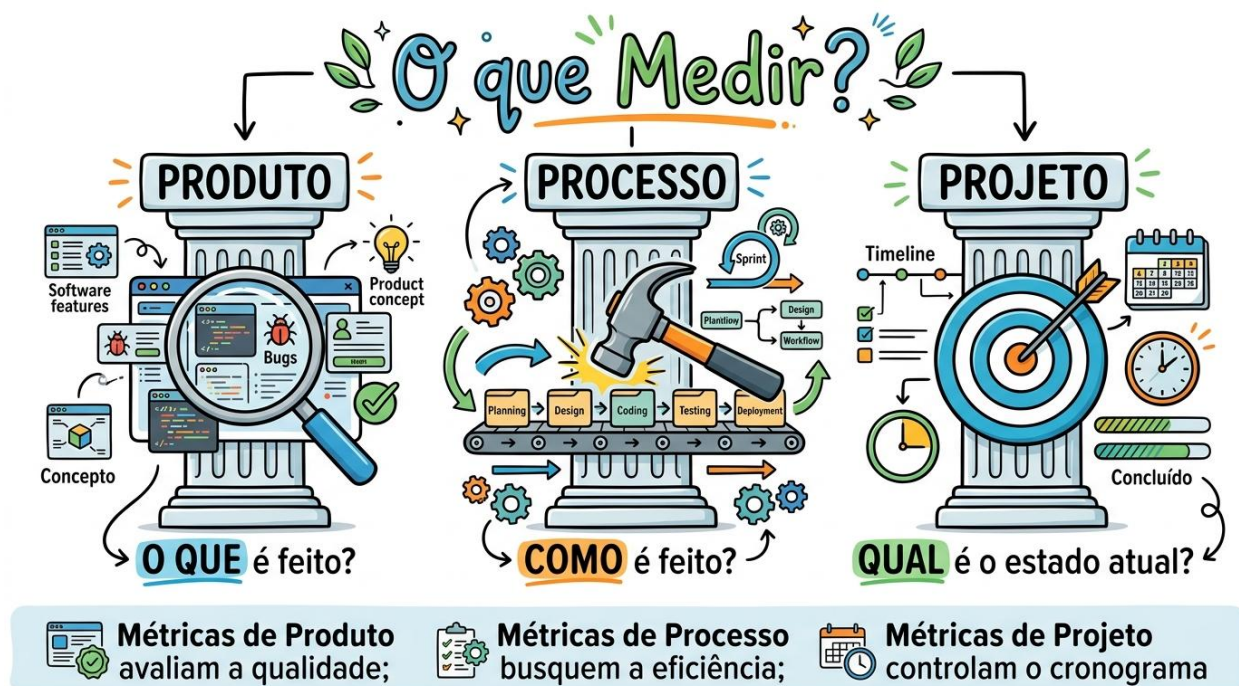
Quanto ao Objeto

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

A Engenharia de Software utiliza uma grande variedade de métricas para apoiar a gestão da qualidade e o acompanhamento dos projetos. Como organizar tantas medidas diferentes? Uma das classificações mais tradicionais divide as métricas em três categorias principais: métricas de produto, métricas de processo e métricas de projeto. Cada grupo possui objetivos específicos e fornece informações voltadas para diferentes aspectos do desenvolvimento de software.

A classificação entre métricas de produto, processo e projeto demonstra que a medição pode apoiar diferentes níveis de gestão. O que cada categoria procura responder? As métricas de produto ajudam a compreender a qualidade dos artefatos produzidos; as métricas de processo avaliam a eficiência das práticas utilizadas pela organização; e as métricas de projeto fornecem informações para controlar e acompanhar a execução de trabalhos específicos. Em conjunto, essas perspectivas permitem transformar dados em conhecimento útil para a melhoria contínua do software e dos processos de desenvolvimento.

CATEGORIAS	O QUE AVALIA?	OBJETIVO PRINCIPAL	EXEMPLOS DE MÉTRICAS
PRODUTO	Artefatos produzidos durante o desenvolvimento, como requisitos, modelos, arquitetura, código-fonte, documentação e testes.	Avaliar características do software, como qualidade, complexidade, manutenibilidade, desempenho e confiabilidade.	Linhas de Código (LOC), Complexidade Ciclométrica, Fan-In/Fan-Out, Coesão, Acoplamento, Densidade de Defeitos, Tempo de Resposta, Throughput, Uso de Recursos, MTTF, MTBF, Disponibilidade, Índice de Precisão e Revocação.
PROCESSO	Métodos, atividades e práticas utilizados para desenvolver e manter o software.	Avaliar a eficiência, a qualidade e a produtividade dos processos organizacionais, apoiando iniciativas de melhoria contínua.	DRE (Defect Removal Efficiency), MTTR, Cobertura de Testes, Taxa de Detecção de Defeitos, Taxa de Correção de Defeitos, Produtividade, Taxa de Retrabalho, Tempo Médio de Desenvolvimento.
PROJETO	Estado e desempenho de um projeto específico em execução.	Apoiar o planejamento, monitoramento e controle de cronograma, custos, esforço, recursos e riscos.	Quantidade de Mudanças por Versão, Custo do Projeto, Prazo de Desenvolvimento, Quantidade de Riscos Identificados, Taxa de Cumprimento de Marcos, Esforço, Valor Agregado.



Métricas de Produto

Métricas de Tamanho

INCIDÊNCIA EM PROVA: ALTA

As métricas de tamanho procuram quantificar a dimensão de um produto de software por meio de medidas relacionadas à sua extensão funcional ou física. Elas fornecem uma indicação do porte do sistema e servem como base para estimativas de esforço, prazo, custo e produtividade. Entre os exemplos mais comuns estão Linhas de Código (LOC), Pontos de Função, número de módulos, número de classes, número de atributos e número de operações.

Pontos de Função

PONTOS DE FUNÇÃO

Métrica orientada à funcionalidade utilizada para medir o tamanho de um software com base nas funcionalidades entregues ao usuário, independentemente da linguagem de programação, da tecnologia ou da forma de implementação adotada. Seu cálculo considera elementos como entradas externas, saídas externas, consultas externas, arquivos lógicos internos e interfaces externas, permitindo quantificar a complexidade funcional do sistema.

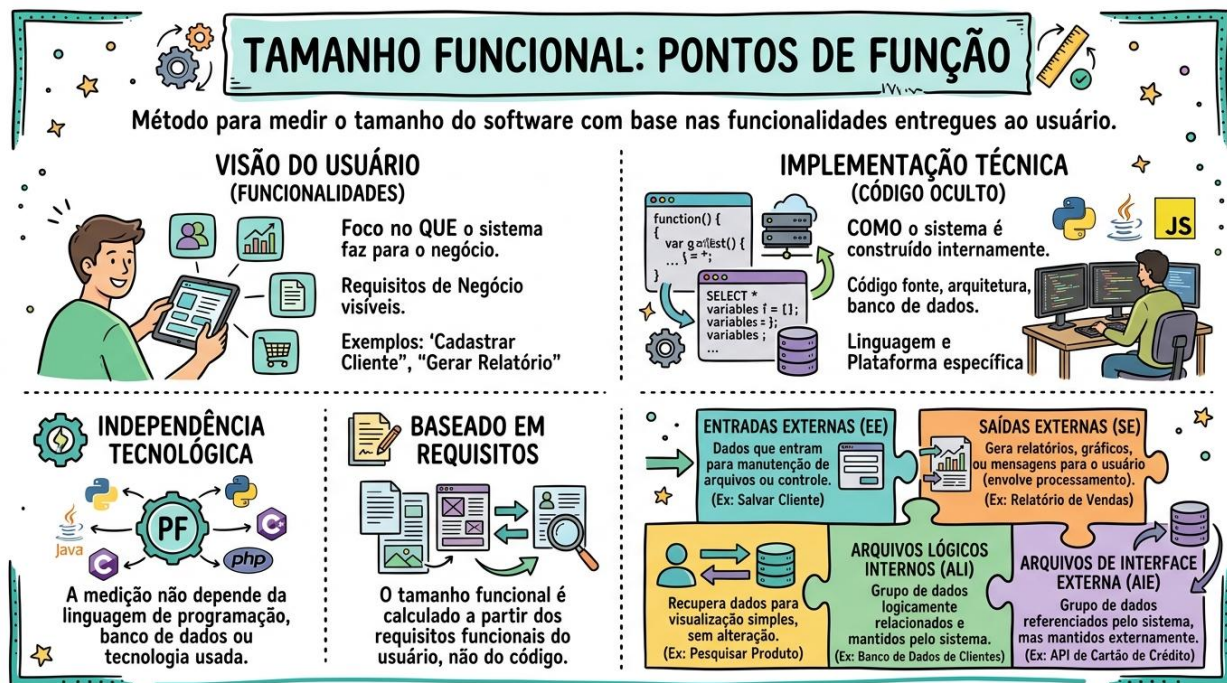
A Análise por Pontos de Função é uma técnica utilizada para medir o tamanho funcional de um software a partir daquilo que o usuário percebe e utiliza. Por que essa abordagem é importante? Porque o objetivo não é avaliar a tecnologia empregada nem a quantidade de código produzida, mas sim as funcionalidades efetivamente disponibilizadas pelo sistema. Dessa forma, a métrica permanece válida mesmo quando diferentes linguagens de programação, plataformas ou arquiteturas são utilizadas para implementar a mesma solução.

Para realizar essa medição, a metodologia considera cinco tipos de componentes funcionais: Entradas Externas, Saídas Externas, Consultas Externas, Arquivos Lógicos Internos e Arquivos de Interface Externa. O que diferencia esses componentes entre si? Cada um representa uma forma distinta de interação ou armazenamento de informações dentro do sistema. Além disso, a metodologia atribui pesos conforme a complexidade funcional identificada em cada componente, seguindo critérios definidos pelas regras da própria técnica.

Uma característica relevante dos Pontos de Função é sua independência em relação à implementação técnica. Imagine dois sistemas desenvolvidos em linguagens completamente diferentes. Eles terão o mesmo tamanho? A resposta depende das funcionalidades oferecidas aos usuários. Se ambos atenderem ao mesmo conjunto de requisitos funcionais, a quantidade de Pontos de Função tende a ser semelhante, ainda que o volume de código, a estrutura interna e as tecnologias utilizadas sejam bastante diferentes.

Essa característica torna a métrica especialmente útil em atividades de planejamento e estimativa. Em que momento ela se torna mais valiosa? Frequentemente nas fases iniciais do projeto, quando ainda não existe software implementado. Como a contagem é baseada nos requisitos funcionais do usuário, torna-se possível estimar prazo, custo, esforço e produtividade antes do início da construção do sistema, desde que o escopo funcional já esteja suficientemente definido.

Por fim, é possível encontrar a métrica de Pontos de Função sendo classificada como uma Métrica de Função ou como uma Métrica de Requisitos.



Pontos de Casos de Uso

PONTOS DE CASOS DE USO

Métrica de tamanho funcional utilizada para estimar o esforço, o custo e o prazo de desenvolvimento de um software a partir de seus casos de uso. Essa técnica considera a quantidade e a complexidade dos atores e dos casos de uso identificados nos requisitos, além de fatores técnicos e ambientais que podem influenciar o projeto, como desempenho, experiência da equipe e restrições tecnológicas.

Os casos de uso descrevem funcionalidades do sistema sob a perspectiva dos usuários e do negócio, sendo definidos nas fases iniciais do desenvolvimento. Por representarem comportamentos esperados do sistema sem depender de detalhes técnicos de implementação, podem ser utilizados como base para estimativas de tamanho, esforço, prazo e custo antes mesmo da construção do software. Essa característica os torna especialmente úteis em atividades de planejamento e gerenciamento de projetos.

Apesar dessa utilidade, a simples contagem de casos de uso apresenta limitações importantes. Nem todos possuem o mesmo nível de complexidade: alguns representam funcionalidades simples, enquanto outros envolvem diversos fluxos alternativos, regras de negócio e interações complexas. Como consequência, comparar sistemas apenas pela quantidade de casos de uso pode gerar estimativas imprecisas, especialmente quando os níveis de detalhamento são diferentes.

Para reduzir essas limitações, foi criada a métrica Pontos de Casos de Uso (Use Case Points - UCP). Essa abordagem considera não apenas a quantidade de casos de uso, mas também a complexidade dos atores, dos próprios casos de uso e de fatores técnicos e ambientais que influenciam o projeto. Dessa forma, os UCP fornecem uma estimativa mais consistente do tamanho funcional do software, permitindo avaliar esforço, prazo e custo ainda nas etapas iniciais do desenvolvimento.



Linhas de Código

LINHAS DE CÓDIGO

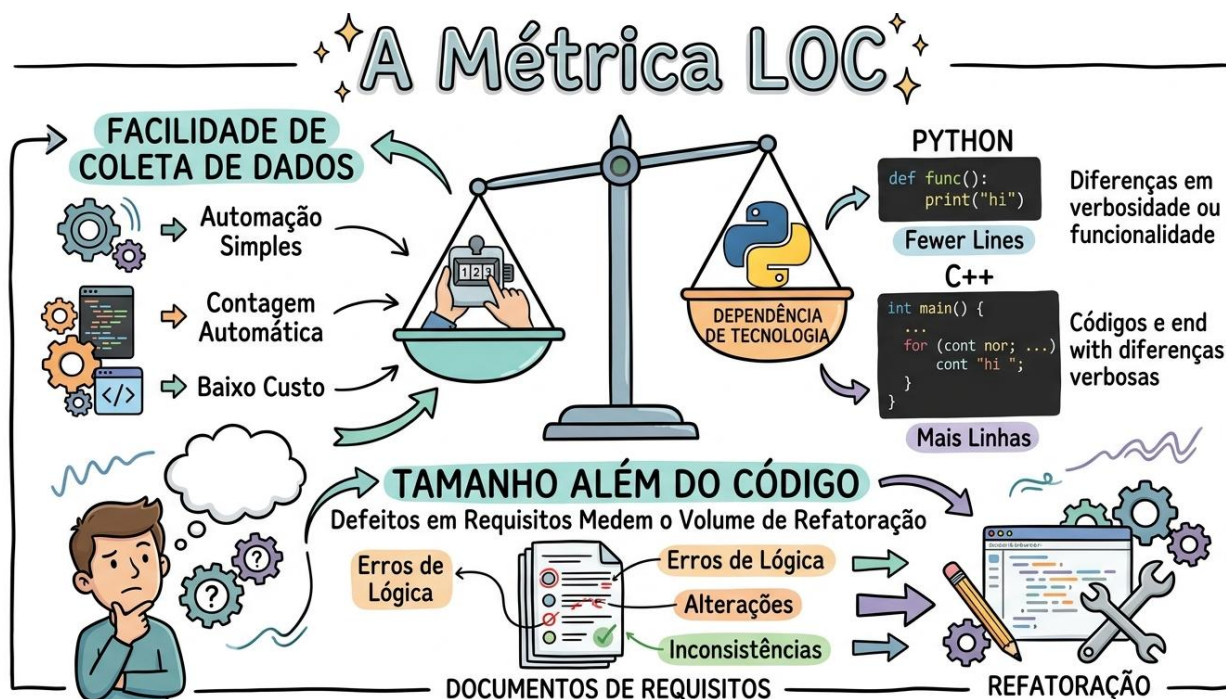
A métrica de Linhas de Código (Lines of Code - LOC) mede o tamanho de um programa por meio da contagem das linhas presentes no código-fonte. Embora não avalie diretamente a qualidade do software, é frequentemente utilizada como indicador de complexidade, esforço de desenvolvimento e propensão a defeitos. Módulos maiores tendem a demandar mais testes e manutenção.

A métrica de Linhas de Código (LOC) é uma das medidas mais tradicionais da Engenharia de Software. Sua popularidade decorre da facilidade de obtenção e da ampla utilização em modelos de estimativa de esforço, custo, produtividade e qualidade. Como praticamente todo projeto produz código-fonte, a LOC tornou-se uma referência histórica importante, permitindo comparações entre projetos e a construção de bases de dados utilizadas em pesquisas e processos de gerenciamento.

Entre suas principais vantagens está o fato de ser uma medida direta, obtida pela simples contagem das linhas presentes no código. Além disso, existe uma grande quantidade de dados históricos baseados nessa métrica, o que facilita análises de produtividade e evolução ao longo do tempo. Por essas razões, a LOC continua sendo amplamente utilizada para avaliar tamanho de software e apoiar indicadores relacionados ao desenvolvimento e à manutenção de sistemas.

Apesar de sua utilidade, a métrica apresenta limitações importantes. A quantidade de linhas necessárias para implementar uma mesma funcionalidade pode variar significativamente entre linguagens de programação diferentes. Além disso, escrever mais código não significa necessariamente produzir uma solução melhor ou mais produtiva. Muitas vezes, projetos bem elaborados resolvem problemas com menos código, utilizando abstrações, reutilização e recursos avançados da linguagem empregada.

Outra restrição da LOC é sua dificuldade de aplicação nas fases iniciais do projeto e em ambientes com forte reutilização, plataformas low-code ou linguagens declarativas. Nesses contextos, a quantidade de código não reflete adequadamente a funcionalidade entregue. Por isso, a LOC deve ser interpretada com cautela e combinada com outras métricas, como Pontos de Função. Também podem ser consideradas métricas de tamanho aplicadas a outros artefatos, como a quantidade de defeitos encontrados em requisitos ou documentos de especificação.



Número de Módulos

O número de módulos é uma métrica de tamanho estrutural que representa a quantidade de componentes, unidades ou partes em que um sistema foi dividido. Dependendo da arquitetura utilizada, um módulo pode corresponder a um pacote, subsistema, serviço, biblioteca, componente ou conjunto coeso de funcionalidades. Essa métrica auxilia na avaliação da decomposição do software, fornecendo uma visão geral de sua organização e distribuição funcional.

Número de Classes

O número de classes é uma métrica de tamanho amplamente utilizada em sistemas orientados a objetos e corresponde à quantidade de classes definidas no projeto ou na implementação do software. As classes representam os principais elementos de modelagem do sistema, encapsulando atributos e operações que descrevem objetos do domínio do problema. Por exemplo, em um sistema acadêmico podem existir classes como Aluno, Professor, Disciplina, Turma e Matrícula. Dessa forma, essa métrica fornece uma indicação do tamanho e da estrutura geral da solução orientada a objetos.

Número de Atributos

O número de atributos é uma métrica amplamente utilizada em projetos orientados a objetos e corresponde à quantidade de dados ou características armazenadas em uma classe. Os atributos representam o estado dos objetos criados a partir dessa classe e descrevem informações relevantes para o domínio do problema. Por exemplo, uma classe Cliente pode possuir atributos como nome, CPF, endereço, telefone e data de nascimento.

Número de Operações

O número de operações corresponde à quantidade de métodos ou funções definidos em uma classe, módulo ou componente de software. Em sistemas orientados a objetos, as operações representam os comportamentos disponibilizados pelos objetos, ou seja, as ações que eles podem executar. Por

exemplo, uma classe ContaBancária pode possuir operações como depositar(), sacar(), transferir() e consultarSaldo().

MÉTRICAS	O QUE MEDE?	O QUE PODE INDICAR?
NÚMERO DE MÓDULOS	Quantidade de componentes, pacotes, serviços, subsistemas ou unidades funcionais em que o software foi dividido.	Avalia o grau de modularização do sistema. Poucos módulos podem indicar concentração de responsabilidades; muitos módulos podem aumentar a complexidade de integração.
NÚMERO DE CLASSES	Quantidade de classes definidas em um projeto orientado a objetos.	Indica o tamanho e a decomposição da solução. Poucas classes podem sugerir baixa modularização; muitas classes podem indicar fragmentação excessiva.
NÚMERO DE ATRIBUTOS	Quantidade de dados ou características armazenadas em uma classe.	Auxilia na avaliação da complexidade e das responsabilidades das classes. Muitos atributos podem indicar excesso de responsabilidades; poucos podem sugerir modelagem simplificada demais.
NÚMERO DE OPERAÇÕES	Quantidade de métodos ou funções definidos em uma classe, módulo ou componente.	Permite avaliar a complexidade funcional e a distribuição de comportamentos. Muitas operações podem indicar classes excessivamente complexas; poucas podem indicar baixa relevância funcional.

Métricas de Estrutura

INCIDÊNCIA EM PROVA: MÉDIA

As métricas estruturais avaliam a organização interna do software, analisando aspectos relacionados à arquitetura, ao projeto e ao código-fonte. Seu objetivo é medir características que influenciam a compreensibilidade, a manutenibilidade e a complexidade do sistema. Exemplos incluem Complexidade Ciclométrica, Fan-In/Fan-Out, acoplamento, coesão, número de módulos e outras métricas que descrevem como os componentes do software estão organizados e interagem entre si.

Complexidade Ciclométrica

COMPLEXIDADE CICLOMÁTICA

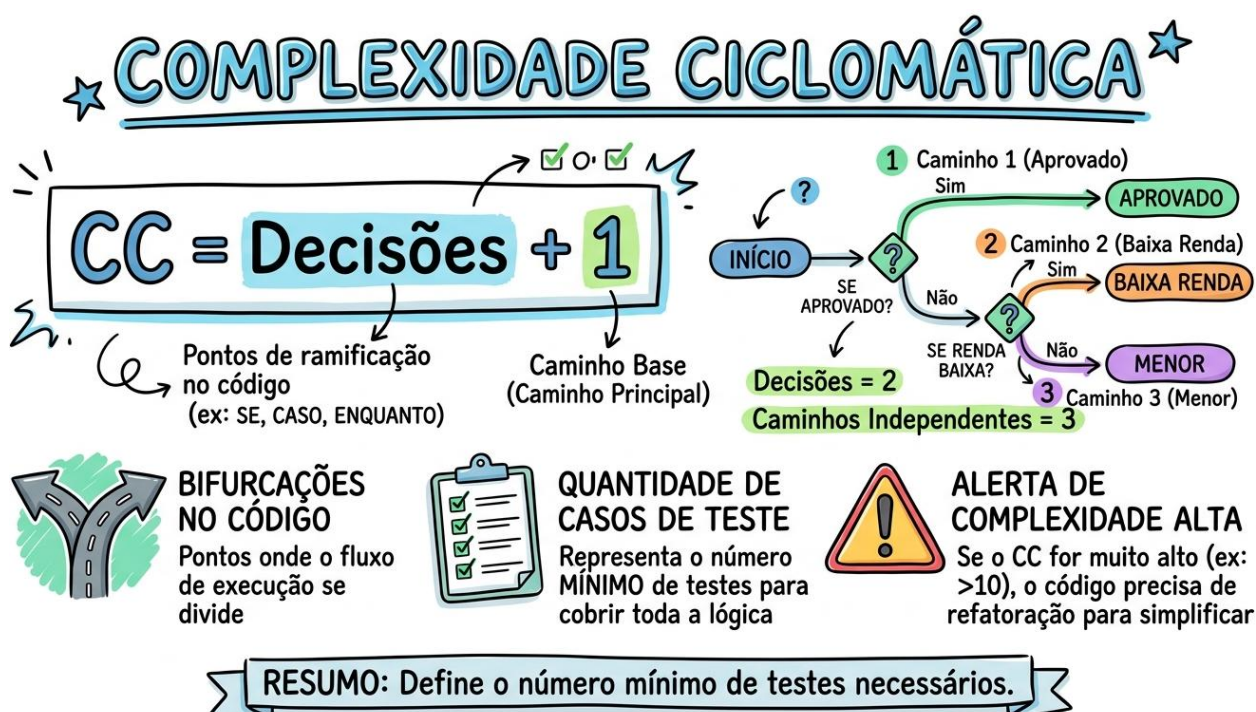
A Complexidade Ciclométrica é uma métrica que mede a complexidade lógica de um programa a partir da quantidade de caminhos independentes de execução existentes em seu fluxo de controle. Quanto maior seu valor, maior tende a ser o esforço necessário para compreender, testar e manter o código. Essa métrica é amplamente utilizada para estimar a quantidade mínima de casos de teste necessários para cobrir todos os caminhos lógicos do software.

A Complexidade Ciclométrica é uma métrica criada por Thomas McCabe para avaliar a complexidade lógica de programas. Baseada na teoria dos grafos, ela estima a dificuldade de compreensão, teste e manutenção do código. Seu principal objetivo é medir a estrutura de controle de um módulo, permitindo identificar componentes potencialmente mais complexos e sujeitos a falhas. Por essa razão, tornou-se uma das métricas estruturais mais utilizadas na Engenharia de Software.

Seu cálculo pode ser realizado a partir do grafo de fluxo de controle por meio da fórmula $V(G) = E - N + 2P$, em que E representa as arestas, N os nós e P os componentes conectados. Em sistemas com apenas um componente conectado, utiliza-se a forma simplificada $V(G) = E - N + 2$. Outra interpretação amplamente difundida considera que a métrica corresponde à quantidade de caminhos linearmente independentes existentes no fluxo de execução do programa.

Em ambientes acadêmicos e concursos, a forma mais comum de cálculo utiliza a expressão $V(G)=D+1$, em que D corresponde ao número de decisões presentes no código. Estruturas como if, else if, for, while, do-while, switch, case e catch aumentam o número de caminhos possíveis e, consequentemente, elevam a Complexidade Ciclomática. Quanto maior a quantidade de desvios de execução, maior tende a ser o valor obtido pela métrica.

A Complexidade Ciclomática possui forte relação com os testes estruturais, pois indica quantos caminhos independentes devem ser considerados no conjunto base de testes. Valores elevados normalmente sinalizam código mais difícil de compreender, testar e manter, além de maior propensão à ocorrência de defeitos. Assim, a métrica auxilia equipes de desenvolvimento a identificar módulos críticos, orientar atividades de refatoração e direcionar esforços para reduzir riscos técnicos ao longo do ciclo de vida do software.



Fan-In/Fan-Out

FAN-IN/FAN-OUT

Fan-In mede o número de módulos ou funções que utilizam um determinado componente, enquanto Fan-Out mede a quantidade de módulos ou funções que são chamados por esse componente. Um Fan-In elevado normalmente indica alto grau de reutilização, enquanto um Fan-Out elevado pode indicar maior complexidade e dependência de outros componentes. Essas métricas ajudam na avaliação da arquitetura e da modularização do sistema.

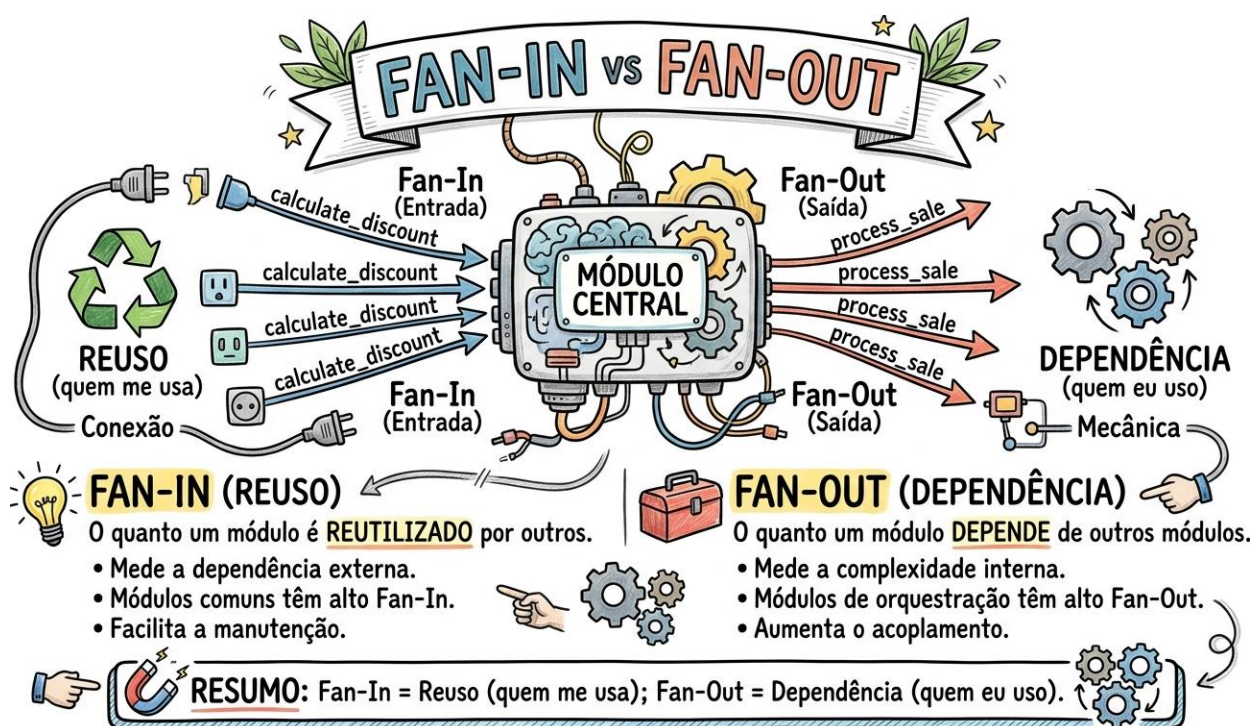
As métricas Fan-In e Fan-Out são utilizadas para analisar as dependências existentes entre componentes de software. Elas ajudam a compreender aspectos relacionados à reutilização, ao acoplamento e à complexidade arquitetural do sistema. Ao avaliar como módulos, funções ou classes se relacionam entre si, essas métricas fornecem informações importantes para atividades de manutenção, testes, análise de impacto e melhoria da qualidade estrutural do software.

O Fan-In mede quantos componentes utilizam determinado módulo. Um valor elevado normalmente indica que o componente é amplamente reutilizado e desempenha papel relevante na arquitetura. Entretanto, como diversos elementos dependem dele, modificações nesse componente podem gerar

impactos significativos em várias partes do sistema. Por isso, módulos com alto Fan-In costumam exigir atenção especial durante manutenção e testes de regressão.

O Fan-Out mede quantos componentes são utilizados por determinado módulo. Quanto maior esse valor, maior tende a ser a quantidade de dependências que precisam ser compreendidas, testadas e mantidas. Em geral, valores elevados podem indicar maior acoplamento e complexidade estrutural. Contudo, a interpretação deve considerar o contexto arquitetural, pois alguns componentes naturalmente precisam interagir com diversos serviços ou módulos para cumprir suas responsabilidades.

A análise conjunta de Fan-In e Fan-Out fornece uma visão mais completa da estrutura interna do software. Enquanto o Fan-In identifica componentes amplamente reutilizados e potencialmente críticos, o Fan-Out destaca módulos com elevado número de dependências. Essas métricas demonstram que a qualidade estrutural não depende apenas do tamanho do código, mas também das relações existentes entre seus componentes, influenciando diretamente a manutenção, a evolução e a confiabilidade do sistema.



Coesão e Acoplamento

COESÃO E ACOPLAMENTO

Coesão mede o grau de relacionamento entre os elementos internos de um módulo ou classe, enquanto acoplamento mede o nível de dependência entre módulos distintos. Em projetos de software, busca-se obter alta coesão e baixo acoplamento, pois essa combinação favorece a manutenção, a reutilização e a evolução do sistema. Esses atributos são amplamente utilizados na avaliação da qualidade de projetos orientados a objetos.

O acoplamento e a coesão são atributos fundamentais da qualidade estrutural de software, pois influenciam diretamente a manutenção, os testes e a evolução dos sistemas. Enquanto o acoplamento avalia as dependências existentes entre componentes, a coesão analisa o grau de relacionamento entre os elementos internos de um mesmo módulo. A combinação adequada desses atributos contribui para a construção de sistemas mais organizados, compreensíveis e fáceis de modificar ao longo do tempo.

O acoplamento mede o quanto um componente depende de outros para executar suas funções. Dependências excessivas tendem a aumentar a complexidade do software, pois alterações em um

módulo podem gerar impactos inesperados em diversas partes da aplicação. Embora algum nível de dependência seja inevitável, busca-se minimizar relações desnecessárias para reduzir riscos, facilitar a manutenção e tornar os componentes mais independentes e reutilizáveis.

A coesão avalia o quanto os elementos internos de um componente trabalham juntos para cumprir uma responsabilidade bem definida. Componentes altamente coesos possuem objetivos claros e comportamento mais previsível, enquanto baixa coesão geralmente indica acúmulo de responsabilidades distintas em um mesmo módulo. Por isso, um dos princípios clássicos de projeto é buscar alta coesão e baixo acoplamento, favorecendo sistemas mais simples de compreender, testar, reutilizar e evoluir.



Métricas de Confiabilidade

INCIDÊNCIA EM PROVA: BAIXA

As métricas de confiabilidade medem a capacidade do software de operar corretamente durante um determinado período de tempo, sem apresentar falhas ou interrupções. Essas métricas são amplamente utilizadas para avaliar a estabilidade e a disponibilidade de sistemas em produção, fornecendo informações importantes sobre sua robustez operacional. Entre os exemplos mais comuns estão MTTF (Mean Time To Failure), MTBF (Mean Time Between Failures), Disponibilidade e Taxa de Falhas.

MTTF (Mean Time To Failures)

MTTF (MEAN TIME TO FAILURES)

O MTTF representa o Tempo Médio até a Falha e mede quanto tempo, em média, um sistema opera antes que ocorra uma falha. Essa métrica é amplamente utilizada para avaliar a confiabilidade de sistemas e componentes. Quanto maior o MTTF, maior tende a ser a capacidade do software de funcionar continuamente sem interrupções.

Entre essas métricas destaca-se o MTTF (Mean Time To Failures). O que essa medida representa? O MTTF corresponde ao tempo médio de operação até a ocorrência de uma falha. Seu cálculo é realizado

dividindo-se o tempo total de funcionamento pelo número de falhas registradas durante o período analisado. Se um sistema operar durante mil horas e apresentar cinco falhas, por exemplo, o resultado será um MTTF de duzentas horas. Isso indica que, em média, o sistema permanece operacional durante esse intervalo antes da ocorrência de uma nova falha.

O MTTF costuma ser utilizado quando o interesse principal está relacionado à frequência das falhas observadas. Em quais situações essa métrica é mais útil? Ela é frequentemente empregada na avaliação da estabilidade de versões de software, na comparação entre produtos e na análise da evolução da qualidade ao longo do tempo. Valores mais elevados indicam períodos maiores de funcionamento contínuo, sugerindo maior estabilidade operacional.

MTBF (Mean Time Between Failures)

MTBF (MEAN TIME BETWEEN FAILURES)

O MTBF (Mean Time Between Failures) representa o Tempo Médio Entre Falhas e mede o intervalo médio de operação entre uma falha e a próxima em sistemas reparáveis. Trata-se de uma das métricas mais utilizadas para avaliar confiabilidade e disponibilidade. Valores elevados indicam menor frequência de falhas e maior estabilidade operacional.

O MTBF (Mean Time Between Failures) é uma métrica de confiabilidade utilizada em sistemas reparáveis para medir o tempo médio entre falhas sucessivas. Diferentemente do MTTF, que considera apenas o tempo médio até a ocorrência de uma falha, o MTBF incorpora também o período necessário para restaurar o sistema. Por isso, costuma ser expresso pela relação $MTBF = MTTF + MTTR$, oferecendo uma visão mais abrangente do comportamento operacional do serviço.

A análise conjunta de MTTF, MTTR e MTBF fornece informações valiosas para a gestão da qualidade e da manutenção. Enquanto o MTTF indica a frequência com que as falhas ocorrem, o MTTR mede o tempo médio necessário para corrigir os problemas. Já o MTBF representa o intervalo médio entre interrupções sucessivas, permitindo avaliar a estabilidade operacional do sistema. Em geral, valores mais elevados de MTTF e MTBF indicam maior confiabilidade e continuidade dos serviços.

Para acompanhar a qualidade operacional dos serviços, as organizações utilizam os Service Level Indicators (SLI), que são métricas objetivas de desempenho. Os SLI servem de base para verificar o cumprimento dos Service Level Objectives (SLO), que representam metas desejadas para essas métricas. Já os Service Level Agreements (SLA) formalizam contratualmente esses compromissos. Assim, os SLI fornecem os dados necessários para avaliar se os objetivos e acordos de nível de serviço estão sendo efetivamente atendidos.

SIGLA	NOME	O QUE É?	EXEMPLO
SLI	Service Level Indicator (Indicador de Nível de Serviço)	Métrica utilizada para medir o desempenho ou a qualidade de um serviço.	Disponibilidade medida em 99,92%; tempo médio de resposta de 250 ms.
SLO	Service Level Objective (Objetivo de Nível de Serviço)	Meta estabelecida para um ou mais SLIs. Define o nível de qualidade desejado para o serviço.	Disponibilidade mínima de 99,9% ao mês; tempo de resposta inferior a 300 ms em 95% das requisições.
SLA	Service Level Agreement (Acordo de Nível de Serviço)	Contrato formal entre provedor e cliente que estabelece os SLOs e as consequências caso não sejam cumpridos.	Contrato prevendo disponibilidade de 99,9% e desconto na fatura caso a meta não seja atingida.

A Disponibilidade é uma métrica que mede o percentual de tempo em que um sistema permanece operacional e acessível aos usuários. Muito utilizada na gestão de serviços de TI, ela permite avaliar a

continuidade operacional e verificar o cumprimento de acordos de nível de serviço (SLAs). Embora frequentemente associada ao termo Uptime, a Disponibilidade normalmente é expressa como uma porcentagem calculada pela relação entre o tempo disponível e o tempo total observado.

Essa métrica é especialmente importante porque reflete diretamente a experiência dos usuários. Um sistema pode apresentar bom desempenho e poucos defeitos, mas ainda causar insatisfação se ficar indisponível com frequência. Por isso, a Disponibilidade é amplamente utilizada em contratos de hospedagem, nuvem e suporte técnico. Além disso, ela complementa métricas de confiabilidade, fornecendo uma visão mais abrangente da qualidade operacional e da capacidade do serviço de permanecer acessível quando necessário.

MTTR (Mean Time To Repair)

MTTR (MEAN TIME TO REPAIR)

O MTTR representa o Tempo Médio de Reparo e mede o tempo necessário para corrigir uma falha e restaurar o funcionamento normal do sistema. Essa métrica está diretamente relacionada à manutenibilidade e à capacidade de recuperação do software. Quanto menor o MTTR, mais rapidamente o sistema retorna à operação após uma falha.

O MTTR (Mean Time To Repair), ou Tempo Médio para Reparo, é uma métrica utilizada para medir o tempo médio necessário para restaurar um sistema após a ocorrência de uma falha. Seu objetivo é avaliar a rapidez com que a organização consegue recuperar os serviços interrompidos, considerando atividades como diagnóstico, correção do problema e retorno à operação normal. Trata-se de um importante indicador da eficiência operacional das equipes de suporte e manutenção.

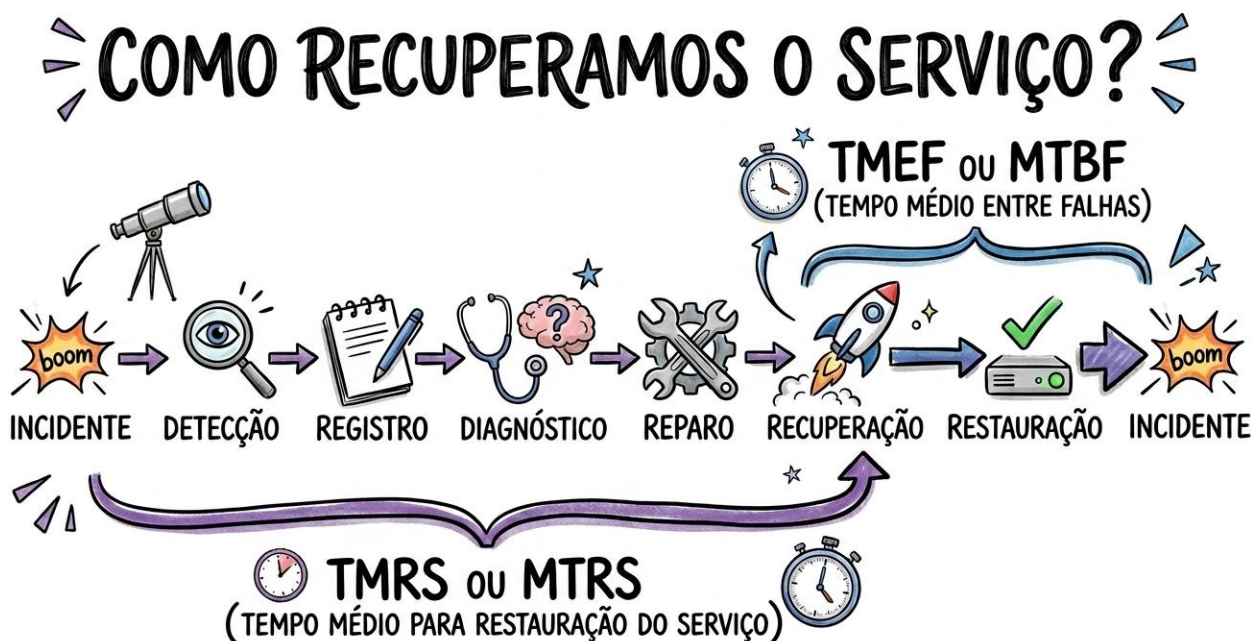
O cálculo do MTTR é realizado pela divisão do tempo total gasto nos reparos pela quantidade de falhas ocorridas em determinado período. A fórmula é dada por $MTTR = \text{Tempo Total de Reparo} \div \text{Número de Falhas}$. O resultado obtido representa o tempo médio necessário para restaurar o funcionamento do sistema após cada incidente, permitindo acompanhar a eficiência dos processos de recuperação ao longo do tempo.

Essa métrica possui grande relevância porque está diretamente relacionada ao impacto das falhas sobre os usuários e sobre os processos de negócio. Quanto menor for o MTTR, menor tende a ser o período de indisponibilidade dos serviços e menores serão os prejuízos causados por interrupções. Por isso, organizações investem em monitoramento, automação, procedimentos de contingência e capacitação das equipes para reduzir continuamente esse indicador.

É importante destacar que o MTTR não mede a frequência das falhas nem a confiabilidade do software. Esses aspectos são normalmente avaliados por métricas como MTTF e MTBF. O MTTR está associado à capacidade de recuperação do sistema, indicando a rapidez com que a organização consegue reagir quando um problema já ocorreu. Assim, ele complementa as métricas de confiabilidade ao fornecer uma visão da eficiência dos processos de manutenção.

A evolução do MTTR também pode revelar o grau de maturidade operacional da organização. Reduções consistentes nesse indicador costumam refletir melhorias nos processos de suporte, diagnóstico e correção de incidentes. Embora possa ser associado à confiabilidade por influenciar a indisponibilidade dos serviços, o MTTR é geralmente classificado pela ISO 25010 como uma métrica de manutenibilidade, pois avalia a facilidade e a rapidez com que o software pode ser reparado após a ocorrência de falhas.

MTTF x MTBF x MTTR



A imagem anterior ajuda a entender o ciclo completo de tratamento de uma falha em um serviço de tecnologia da informação. O processo começa com a ocorrência de um incidente, que provoca uma interrupção ou degradação do serviço. Em seguida, a falha é detectada, registrada formalmente e analisada durante a etapa de diagnóstico, na qual a equipe busca identificar sua causa. Após a identificação do problema, são realizadas as atividades de reparo, seguidas da recuperação e da restauração do serviço, retornando o sistema ao funcionamento normal.

A imagem também destaca a métrica MTRS (Mean Time to Restore Service), que representa o Tempo Médio para Restauração do Serviço (TMRS). Essa métrica mede todo o período compreendido entre a ocorrência do incidente e a efetiva restauração do serviço aos usuários. Logo, ela engloba não apenas o tempo de reparo, mas também as etapas de detecção, registro, diagnóstico, recuperação e validação da solução. Quanto menor for o TMRS, mais eficiente tende a ser a capacidade da organização em responder e recuperar-se de falhas operacionais.

Outro conceito apresentado é o MTBF (Mean Time Between Failures), conhecido como Tempo Médio Entre Falhas (TMEF). Essa métrica mede o intervalo médio entre a ocorrência de duas falhas consecutivas em um sistema. Na imagem, o MTBF corresponde ao período de operação normal compreendido entre a restauração do serviço após uma falha e a ocorrência do próximo incidente. Valores elevados de MTBF indicam que o sistema consegue permanecer funcionando por mais tempo sem interrupções, sugerindo maior confiabilidade operacional.

Por fim, a imagem evidencia que a disponibilidade de um serviço depende tanto da rapidez na recuperação quanto da frequência das falhas. Um sistema considerado confiável deve apresentar um MTBF elevado, indicando longos períodos de funcionamento sem incidentes, e um TMRS reduzido, demonstrando capacidade de restaurar rapidamente os serviços quando ocorrem problemas. Dessa forma, essas métricas são amplamente utilizadas em gestão de serviços, operações de TI e monitoramento de infraestrutura para avaliar a qualidade, a estabilidade e a disponibilidade dos sistemas em produção.

Métricas de Processo

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

DEFECT REMOVAL EFFICIENCY (DRE)

O Defect Removal Efficiency (DRE) mede a eficiência do processo de desenvolvimento e testes na identificação e remoção de defeitos antes da entrega do software. Seu cálculo compara os defeitos eliminados durante o desenvolvimento com o total de defeitos encontrados antes e após a entrega. Valores elevados indicam maior eficácia das atividades de verificação e validação.

MÉTRICAS	O QUE MEDE?	O QUE INDICA?
TAXA DE RETRABALHO	Percentual do esforço gasto na correção, revisão ou repetição de atividades já realizadas.	Níveis elevados podem indicar problemas de requisitos, comunicação, projeto ou qualidade da implementação.
PRODUTIVIDADE	Relação entre o tamanho do software produzido e o esforço consumido para produzi-lo.	Permite avaliar a eficiência da equipe e apoiar estimativas de prazo, custo e esforço em projetos futuros.
MUDANÇAS POR VERSÃO	Quantidade de alterações incorporadas em cada nova versão do software.	Indica o ritmo de evolução do sistema e o volume de correções, melhorias e novas funcionalidades implementadas.

Métricas de Projeto

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

As métricas de projeto são medidas quantitativas utilizadas para monitorar e controlar a execução de um projeto de software, fornecendo informações sobre aspectos como esforço, custo, prazo, produtividade, utilização de recursos e riscos. Diferentemente das métricas de produto e de processo, seu foco está no gerenciamento do projeto em andamento. Elas permitem comparar o desempenho real com o planejado, identificar desvios, antecipar problemas e apoiar a tomada de decisões, contribuindo para aumentar as chances de sucesso do empreendimento.

MÉTRICAS	O QUE MEDE?	O QUE INDICA?
ESFORÇO	Quantidade de trabalho necessária para desenvolver, manter ou evoluir o software, geralmente medida em horas, dias ou pessoas-mês.	Permite avaliar a carga de trabalho da equipe, comparar estimativas com resultados reais e apoiar o planejamento de recursos.
CUSTO	Recursos financeiros consumidos pelo projeto, incluindo mão de obra, infraestrutura, ferramentas, treinamentos, licenças e demais despesas.	Auxilia no controle orçamentário, na análise de viabilidade econômica e na identificação de desvios financeiros.
PRAZO	Tempo necessário para concluir atividades, fases ou o projeto como um todo.	Permite acompanhar o cronograma, identificar atrasos ou antecipações e apoiar a gestão de riscos relacionados às entregas.
VALOR AGREGADO (EARNED VALUE)	Valor orçado correspondente ao trabalho efetivamente concluído em determinado momento do projeto.	Fornece uma visão integrada de escopo, prazo e custo, servindo de base para indicadores de desempenho como SPI e CPI.

QUESTÕES COMENTADAS

1. (VUNESP / Câmara Municipal de Fernandópolis - 2022) Diversas métricas são comumente utilizadas para quantificar o gerenciamento de falhas de um ambiente computacional. Dentre elas, destacam-se o MTTR, MTBF e MTTF. Um ambiente que possui bom gerenciamento de suas falhas é aquele cujas métricas possuam o seguinte:

- a) Alto MTTR, baixo MTBF e baixo MTTF.
- b) Alto MTTR, baixo MTBF e alto MTTF.
- c) Alto MTTR, alto MTBF e alto MTTF.
- d) Baixo MTTR, baixo MTBF e alto MTTF.
- e) Baixo MTTR, baixo MTBF e baixo MTTF.

Comentários:

Idealmente, uma organização deve possuir:

- Baixo Tempo Médio de Reparação (MTTR), de modo que falhas sejam corrigidas rapidamente;
- Alto Tempo Médio Entre Falhas (MTBF), de modo que ocorram raramente;
- Alto Tempo Médio Para Falhas (MTTF), de modo que o sistema funcione bastante sem falhar.

Dito isso, não temos alternativa correta na questão e ela deveria ter sido anulada.

Gabarito: Letra D

2. (VUNESP / MPE ES - 2013) Uma das métricas utilizadas no desenvolvimento de software é denominada fan-out, que representa o

- a) número de instruções de desvio incondicional do módulo sob medição.
- b) número de linhas de código do módulo sob medição.
- c) número de módulos acionados pelo módulo sob medição.
- d) tempo máximo de execução do módulo sob medição.
- e) tempo médio de execução do módulo sob medição

Comentários:

(a) Errado. Fan-out não mede quantidade de desvios incondicionais no código, e sim o grau de acionamento entre módulos.

- (b) Errado. Número de linhas de código é outra métrica, ligada ao tamanho do módulo, não ao fan-out.
- (c) Correto. Fan-out representa quantos módulos são chamados ou acionados pelo módulo que está sendo analisado.
- (d) Errado. Tempo máximo de execução está ligado a desempenho, não à métrica fan-out.
- (e) Errado. Tempo médio de execução também mede desempenho, e não a quantidade de módulos acionados.

Gabarito: Letra C

3. (VUNESP / EsFCEx - 2022) Acoplamento e coesão são dois critérios qualitativos utilizados para avaliar a independência funcional de um software, sendo correto que

- a) um módulo coeso requer pouca interação com outros módulos do software.
- b) módulos coesos devem ter um máximo de 200 linhas de código de software.
- c) um módulo coeso deve implementar pelo menos três tarefas do software.
- d) recomenda-se que o acoplamento entre módulos de software seja alto.
- e) o acoplamento limita o número de variáveis contidas em um módulo de software.

Comentários:

- (a) Correto. Coesão alta indica que o módulo é bem focado na sua função, o que tende a reduzir a dependência e a interação com outros módulos.
- (b) Errado. Coesão não é medida por quantidade de linhas de código, mas pela relação funcional entre os elementos do módulo.
- (c) Errado. Um módulo coeso não precisa executar várias tarefas; na verdade, a ideia é justamente manter foco em uma responsabilidade bem definida.
- (d) Errado. Em projeto de software, o ideal é baixo acoplamento, para facilitar manutenção, teste e reaproveitamento dos módulos.
- (e) Errado. Acoplamento trata da dependência entre módulos, não da quantidade de variáveis internas existentes em cada módulo.

Gabarito: Letra A

4. (VUNESP / TJM SP - 2021) Considerando-se as métricas de processo de software, é correto afirmar que um de seus objetivos é

- a) avaliar o estado de um projeto em andamento.
- b) obter indicadores que levem ao aperfeiçoamento de processos do software.
- c) ajustar o fluxo de trabalho das equipes de desenvolvimento.
- d) avaliar a capacidade da equipe de projeto.
- e) detectar áreas problemáticas no projeto.

Comentários:

(a) Errado. Avaliar o estado de um projeto em andamento está mais ligado ao acompanhamento do projeto do que ao foco principal das métricas de processo.

(b) Correto. Métricas de processo servem justamente para gerar indicadores que ajudem a melhorar e aperfeiçoar os processos de desenvolvimento de software.

(c) Errado. Ajustar o fluxo de trabalho pode até ser uma consequência, mas não é o objetivo central das métricas de processo.

(d) Errado. Avaliar a capacidade da equipe diz mais respeito a desempenho de pessoas do que à medição e melhoria do processo.

(e) Errado. Detectar áreas problemáticas no projeto se relaciona mais ao controle do projeto, não ao objetivo principal das métricas de processo.

Gabarito: Letra B

LISTA DE QUESTÕES

- 1. (VUNESP / Câmara Municipal de Fernandópolis - 2022) Diversas métricas são comumente utilizadas para quantificar o gerenciamento de falhas de um ambiente computacional. Dentre elas, destacam-se o MTTR, MTBF e MTTF. Um ambiente que possui bom gerenciamento de suas falhas é aquele cujas métricas possuam o seguinte:**
 - a) Alto MTTR, baixo MTBF e baixo MTTF.
 - b) Alto MTTR, baixo MTBF e alto MTTF.
 - c) Alto MTTR, alto MTBF e alto MTTF.
 - d) Baixo MTTR, baixo MTBF e alto MTTF.
 - e) Baixo MTTR, baixo MTBF e baixo MTTF.

- 2. (VUNESP / MPE ES - 2013) Uma das métricas utilizadas no desenvolvimento de software é denominada fan-out, que representa o**
 - a) número de instruções de desvio incondicional do módulo sob medição.
 - b) número de linhas de código do módulo sob medição.
 - c) número de módulos acionados pelo módulo sob medição.
 - d) tempo máximo de execução do módulo sob medição.
 - e) tempo médio de execução do módulo sob medição

- 3. (VUNESP / EsFCEEx - 2022) Acoplamento e coesão são dois critérios qualitativos utilizados para avaliar a independência funcional de um software, sendo correto que**
 - a) um módulo coeso requer pouca interação com outros módulos do software.
 - b) módulos coesos devem ter um máximo de 200 linhas de código de software.
 - c) um módulo coeso deve implementar pelo menos três tarefas do software.
 - d) recomenda-se que o acoplamento entre módulos de software seja alto.
 - e) o acoplamento limita o número de variáveis contidas em um módulo de software.

- 4. (VUNESP / TJM SP - 2021) Considerando-se as métricas de processo de software, é correto afirmar que um de seus objetivos é**
 - a) avaliar o estado de um projeto em andamento.
 - b) obter indicadores que levem ao aperfeiçoamento de processos do software.
 - c) ajustar o fluxo de trabalho das equipes de desenvolvimento.
 - d) avaliar a capacidade da equipe de projeto.
 - e) detectar áreas problemáticas no projeto.

GABARITO

- | | | | |
|----|---------|----|---------|
| 1. | LETRA D | 3. | LETRA A |
| 2. | LETRA C | 4. | LETRA B |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.